# Payment Reliability Engine (PRE)
## Software Requirements Document
**Version:** 1.0
**Status:** Draft
**Author:** Jason Dagana
**Date:** June 2026
**Classification:** Confidential

---

# 1. Executive Summary

Payment infrastructure is broken at the implementation layer.

Every developer who integrates a payment provider — Stripe, Paystack, Flutterwave, Monnify — in

The Payment Reliability Engine (PRE) is an open-source infrastructure library that sits betwee

PRE is the reliability layer that payment providers do not build and developers cannot afford t

---

# 2. The Problem

## 2.1 What Developers Actually Face

When a developer integrates a payment provider today, the SDK gives them three things: initiali

That means every team, on every project, manually implements:

```
Idempotency           → prevent duplicate charges
Webhook verification  → reject tampered events
Retry logic           → recover from transient failures
State management      → track where a payment is in its lifecycle
Audit logging         → know what happened and when
Reconciliation        → detect mismatches between app and provider
Failure classification → know whether to retry or give up
```

Most teams implement none of these correctly. Some implement a few. Almost no team implements a

## 2.2 The Consequences

The failures that result from this are not minor:

```
Duplicate charges
  Customer is charged twice for the same order
  Chargeback filed → provider fee charged → customer lost

Order fulfillment on failed payments
  Race condition causes fulfillment before verification completes
  Goods shipped or access granted for free

Payment succeeds but application shows failure
  Webhook lost or misprocessed
  Customer support ticket created
  Manual intervention required
  Trust eroded

Retries never happen
  Transient network failure causes permanent payment failure
  Revenue lost that could have been recovered

No audit trail
  Payment dispute raised
```

```
  No record of what happened and when
  Support team cannot investigate
  Business loses the dispute by default
```

## 2.3 Why This Has Not Been Solved

Payment providers have no incentive to solve this. They make money when payments process — not

Libraries that wrap payment providers exist but they only simplify the API surface. They do not

No library currently exists that treats payment reliability as the primary concern — not the pr

PRE is that library.

---

# 3. Product Vision

PRE is the standard reliability layer for payment infrastructure in the same way that connectio

Developers should not think about duplicate webhook protection any more than they think about T

```
Without PRE:
  Developer calls provider SDK
  Developer manually handles every failure mode
  Developer gets some of them right
  Production failures occur

With PRE:
  Developer calls PRE
  PRE handles every failure mode by default
  Developer focuses on business logic
  Production failures are eliminated or automatically recovered
```

---

# 4. Goals

## 4.1 Primary Goals

```
G1  Eliminate duplicate transaction processing
G2  Enforce valid payment state transitions
G3  Provide unified provider abstraction
G4  Guarantee webhook verification and deduplication
G5  Automate failure recovery through intelligent retry
G6  Produce complete audit trails for every payment
G7  Enable reconciliation between application and provider state
G8  Support extensibility without modifying core
```

## 4.2 Secondary Goals

```
G9   Minimize integration time to under 30 minutes
G10  Provide observable, debuggable payment workflows
G11  Support horizontal scaling without race conditions
G12  Deliver production-ready defaults with zero configuration
```

## 4.3 Non-Goals

PRE explicitly does not:
```
```

Process card payments directly
Hold or move customer funds
Replace payment providers
Store PCI-sensitive card data
Perform KYC or AML operations
Become a payment gateway
Enforce business rules (fulfillment, access control)
```

---

# 5. Target Users

## 5.1 Primary Users

**Backend Developers**
Building payment flows for the first time or the tenth time. Tired of solving the same reliabil

**Full-Stack Developers at Startups**
Moving fast and cannot afford to spend three weeks building a correct payment implementation. N

**SaaS Companies**
Running subscription billing, usage-based pricing, or one-time purchases. Cannot afford billing

**E-commerce Platforms**
Processing high volumes of transactions where duplicate charges or missed webhooks directly imp

## 5.2 Secondary Users

**Fintech Startups**
Building on top of payment infrastructure where correctness is a regulatory and business requir

**Marketplace Builders**
Handling payments between buyers and sellers where split payment logic and reconciliation are c

**Agencies**
Building payment flows for clients and needing a standard, reliable implementation they can dro

---

# 6. Core Concepts

## 6.1 Payment Provider

An adapter that wraps a specific payment gateway and exposes a unified interface to the engine.

```ts
interface PaymentProvider {
  initializePayment(params: InitializeParams): Promise<InitializeResult>;
  verifyPayment(reference: string): Promise<VerifyResult>;
  refundPayment(reference: string, amount?: number): Promise<RefundResult>;
  verifyWebhook(payload: unknown, signature: string): Promise<WebhookEvent>;
  getTransaction(reference: string): Promise<ProviderTransaction>;
}
```

Supported at launch:
- Paystack
- Stripe
- Flutterwave

Planned:
- Monnify
- LemonSqueezy
- Paddle

## 6.2 Transaction
```

The core entity representing a single payment attempt. Every payment that flows through PRE is

```ts
interface Transaction {
  id: string;
  reference: string;
  amount: number;
  currency: string;
  status: TransactionStatus;
  provider: string;
  idempotencyKey: string;
  metadata: Record<string, unknown>;
  createdAt: Date;
  updatedAt: Date;
}
```

## 6.3 Payment State Machine

The state machine is the enforcement layer for valid payment lifecycle transitions. It is the m

```
CREATED     → payment record exists, not yet sent to provider
PENDING     → sent to provider, awaiting user action
PROCESSING  → user has acted, provider is processing
SUCCESS     → provider confirmed payment
FAILED      → provider rejected or timeout occurred
RETRYING    → engine is attempting recovery
REFUNDED    → payment successfully reversed
CANCELLED   → payment abandoned before completion
DISPUTED    → chargeback or dispute raised
```

Valid transitions:

```
CREATED    → PENDING
PENDING    → PROCESSING
PENDING    → CANCELLED
PROCESSING → SUCCESS
PROCESSING → FAILED
FAILED     → RETRYING
RETRYING   → SUCCESS
RETRYING   → FAILED
SUCCESS    → REFUNDED
SUCCESS    → DISPUTED
```

Forbidden transitions (enforced by the engine, throws on attempt):

```
SUCCESS    → PENDING
SUCCESS    → PROCESSING
REFUNDED   → SUCCESS
CANCELLED  → any state
DISPUTED   → SUCCESS
```

## 6.4 Idempotency Key

A unique key supplied by the developer (or generated by PRE) that identifies a specific payment

## 6.5 Webhook Event

A provider-sent notification about a payment state change. PRE verifies the signature, checks f

## 6.6 Retry Record

Metadata about a retry attempt — the reason for the original failure, the retry strategy, the r

## 6.7 Audit Log Entry

An immutable record of every action taken on a transaction — state transitions, webhook events

---

# 7. Functional Requirements

## FR-1 Provider Abstraction

The engine shall expose a single, unified API surface regardless of which payment provider is c

```ts
// switch from Paystack to Stripe by changing one line
const engine = new PaymentEngine({
  provider: new StripeProvider({ secretKey: process.env.STRIPE_SECRET_KEY }),
  storage: new PostgresAdapter(db),
});
```

The provider interface shall be publicly documented so developers can implement adapters for ur

---

## FR-2 Payment Initialization

The engine shall provide a unified initialization API that creates a transaction record, genera

```ts
const result = await engine.initialize({
  amount: 5000,
  currency: "NGN",
  email: "user@example.com",
  idempotencyKey: "order_789_attempt_1",
  metadata: { orderId: "order_789", userId: "user_123" },
});

// result
{
  transactionId: "txn_abc123",
  checkoutUrl: "https://checkout.paystack.com/...",
  reference: "ref_xyz789",
  status: "PENDING",
}
```

If the same idempotency key is submitted while a transaction is in PENDING or PROCESSING state,

---

## FR-3 Webhook Processing

The engine shall provide a single webhook handler entry point that:

**Verifies the signature** using the provider-specific algorithm before any other processing oc

**Checks for duplicates** using the event ID before processing. Duplicate events are acknowledg

**Routes the event** through the state machine to validate the transition is permitted.

**Executes the transition** and persists the new state atomically.

**Emits lifecycle events** to registered application handlers after successful processing.

```ts
```

````
// single entry point for all webhook events
app.post("/webhooks/payment", async (req, res) => {
  await engine.handleWebhook({
    payload: req.body,
    signature: req.headers["x-paystack-signature"] as string,
    raw: req.rawBody,
  });
  res.sendStatus(200);
});
```

The handler shall respond to the provider within 200ms regardless of downstream processing time

---

## FR-4 Idempotency

The engine shall enforce idempotency at two levels:

**Request level** — the same idempotency key cannot initialize two separate transactions. The e

**Processing level** — the same webhook event cannot be processed twice. The engine uses the pr

Idempotency records shall be persisted to storage and survive process restarts. In-memory dedup

```ts
// engine enforces this — no application code required
const result1 = await engine.initialize({ idempotencyKey: "key_123", ... });
const result2 = await engine.initialize({ idempotencyKey: "key_123", ... });

// result2 === result1 — same transaction returned, no duplicate charge
```

---

## FR-5 Transaction State Machine

The engine shall enforce valid state transitions. Any attempt to transition a transaction to an

```ts
// engine enforces this internally — application cannot bypass it
await engine.transition(transactionId, "SUCCESS");
// throws InvalidStateTransitionError if current state does not permit SUCCESS
```

State transitions shall be persisted atomically. A transition that fails mid-persistence shall

---

## FR-6 Retry Engine

The engine shall automatically retry transactions that fail due to transient errors.

**Failure classification** — the engine shall classify failures before deciding to retry:

```
Transient failures (retry):
  Network timeouts
  Rate limit responses (429)
  Provider temporary unavailability (503, 504)
  Connection resets

Permanent failures (do not retry):
  Insufficient funds
  Card declined
  Invalid account number
  Fraud rejection
````

  Expired card
```

**Retry strategies** — the engine shall support:

```ts
{
  strategy: "exponential",     // exponential backoff
  initialDelay: 1000,          // 1 second initial delay
  maxDelay: 30000,             // 30 second maximum delay
  multiplier: 2,               // delay doubles each attempt
  maxAttempts: 5,              // maximum retry count
  jitter: true,                // add randomness to prevent thundering herd
}
```

**Custom retry configuration:**

```ts
const engine = new PaymentEngine({
  provider: new PaystackProvider(),
  retry: {
    strategy: "exponential",
    maxAttempts: 3,
    initialDelay: 2000,
  },
});
```

After exhausting all retry attempts, the transaction shall transition to FAILED and a `payment.

---

## FR-7 Distributed Idempotency

The engine shall guarantee idempotency in distributed environments where multiple instances pro

The engine shall use database-level atomic operations — specifically `INSERT ... ON CONFLICT DO

```sql
-- engine handles this internally
INSERT INTO idempotency_keys (key, transaction_id, created_at)
VALUES ($1, $2, NOW())
ON CONFLICT (key) DO NOTHING
RETURNING *;
-- if nothing returned, another instance already processed this key
```

In-memory caching of processed keys is permitted as an optimization but shall not be relied upo

---

## FR-8 Event System

The engine shall expose a typed event system that developers use to subscribe to payment lifecy

```ts
engine.on("payment.initialized", (event) => {
  // send confirmation email
});

engine.on("payment.success", (event) => {
  // fulfill the order
  // send receipt
  // update subscription status
});
engine.on("payment.failed", (event) => {

````
  // notify customer
  // release reserved inventory
});

engine.on("payment.retrying", (event) => {
  // log retry attempt
});

engine.on("refund.initiated", (event) => {
  // notify customer
});

engine.on("refund.completed", (event) => {
  // update order status
  // reverse inventory
});

engine.on("dispute.raised", (event) => {
  // flag account for review
  // notify support team
});
```

Events shall be emitted after state transitions are committed to storage. An event shall not be

---

## FR-9 Audit Logging

The engine shall produce an immutable, append-only audit log for every transaction. Audit recor

Every audit entry shall record:

```ts
interface AuditEntry {
  id: string;
  transactionId: string;
  event: string;
  previousState: TransactionStatus | null;
  newState: TransactionStatus | null;
  actor: "engine" | "webhook" | "developer" | "system";
  metadata: Record<string, unknown>;
  timestamp: Date;
}
```

Events that generate audit entries:

```
Transaction created
Payment initialized with provider
Webhook received
Webhook signature verified
Webhook signature rejected
State transition attempted
State transition completed
State transition rejected (invalid)
Retry scheduled
Retry attempted
Retry succeeded
Retry exhausted
Reconciliation run
Reconciliation mismatch detected
Refund initiated
Refund completed
```
---
````
````

## FR-10 Reconciliation

The engine shall provide reconciliation utilities that compare the application's transaction st

```ts
// reconcile a single transaction
const result = await engine.reconcile(transactionId);

// result
{
  transactionId: "txn_abc123",
  applicationState: "PENDING",
  providerState: "SUCCESS",
  mismatch: true,
  action: "transition_to_success",
  resolvedAt: "2026-06-20T10:30:00Z",
}
```

```ts
// reconcile all transactions in a time window
const results = await engine.reconcileRange({
  from: new Date("2026-06-20T00:00:00Z"),
  to: new Date("2026-06-20T23:59:59Z"),
  status: "PENDING", // only check pending transactions
});
```

Reconciliation shall be available as:
- On-demand via API call
- Scheduled via built-in scheduler
- Triggered by the PRE dashboard

When a mismatch is detected, the engine shall:
1. Log the mismatch to the audit trail
2. Emit a `reconciliation.mismatch` event
3. Optionally auto-resolve if configured to do so
4. Leave the transaction in its current state if auto-resolve is disabled

---

## FR-11 Refund Support

The engine shall expose a unified refund API that works identically across all supported provid

```ts
// full refund
await engine.refund(transactionId);

// partial refund
await engine.refund(transactionId, {
  amount: 2500,
  reason: "Partial cancellation",
});
```

The refund flow shall:
1. Validate the transaction is in a refundable state (SUCCESS)
2. Validate the refund amount does not exceed the original charge
3. Call the provider refund API
4. Transition the transaction to REFUNDED (full) or record a partial refund
5. Emit `refund.initiated` and `refund.completed` events
6. Log all steps to the audit trail

---

## FR-12 Storage Adapter

The engine shall not be coupled to a specific database. Developers shall provide a storage adap

```ts
interface StorageAdapter {
  // transactions
  createTransaction(data: CreateTransactionInput): Promise<Transaction>;
  findTransaction(id: string): Promise<Transaction | null>;
  findTransactionByReference(ref: string): Promise<Transaction | null>;
  updateTransactionStatus(id: string, status: TransactionStatus): Promise<Transaction>;

  // idempotency
  createIdempotencyKey(key: string, transactionId: string): Promise<boolean>;
  findIdempotencyKey(key: string): Promise<string | null>;

  // webhook events
  createWebhookEvent(data: WebhookEventInput): Promise<WebhookEvent>;
  findWebhookEvent(eventId: string): Promise<WebhookEvent | null>;

  // audit logs
  createAuditEntry(data: AuditEntryInput): Promise<AuditEntry>;
  getAuditTrail(transactionId: string): Promise<AuditEntry[]>;

  // retries
  createRetryRecord(data: RetryInput): Promise<RetryRecord>;
  updateRetryRecord(id: string, data: Partial<RetryRecord>): Promise<RetryRecord>;
  getPendingRetries(): Promise<RetryRecord[]>;

  // atomic operations
  withTransaction<T>(fn: (tx: unknown) => Promise<T>): Promise<T>;
}
```

Official adapters provided at launch:
- `@pre/postgres` — PostgreSQL via pg
- `@pre/prisma` — Prisma ORM

Community adapters (documented interface for community contribution):
- MySQL
- MongoDB
- Drizzle ORM

---

# 8. Non-Functional Requirements

## NFR-1 Performance

```
Payment initialization overhead:   < 100ms above provider latency
Webhook processing overhead:        < 200ms above handler execution
State transition execution:         < 50ms
Idempotency key lookup:             < 10ms (with index)
Audit log write:                    < 20ms (async, non-blocking)
```

Audit logging shall be asynchronous and shall not block the main payment flow.

---

## NFR-2 Reliability

```
Idempotency guarantee:     100% — a key shall never produce two transactions
State machine correctness: 100% — invalid transitions shall never succeed
Webhook deduplication:     100% — an event ID shall never be processed twice
Data loss tolerance:       Zero — all state transitions persisted atomically
```

---

## NFR-3 Scalability

The engine shall be stateless at the process level. All state is stored in the persistence laye

The engine shall support:
- Horizontal scaling with no shared in-process state
- Millions of transactions with proper database indexing
- Concurrent webhook processing without race conditions

---

## NFR-4 Security

```
Webhook signatures     → verified before any processing
Secret keys            → never logged, never exposed in error messages
Card data              → never accepted, never stored
Audit logs             → append-only, never modified after creation
Secret rotation        → supported without downtime
```

---

## NFR-5 Developer Experience

```
Time to first payment:      < 30 minutes from npm install
Zero required configuration: production-safe defaults out of the box
TypeScript first:           full type coverage, no any in public API
Error messages:             actionable, include context and resolution hints
Documentation:              every public method documented with examples
```

---

## NFR-6 Maintainability

```
Plugin architecture:    providers are plugins, not core dependencies
Modular packages:       each provider is a separate npm package
Versioned separately:   core and providers have independent release cycles
No circular deps:       strict dependency direction enforced
Test coverage:          > 90% on core state machine and idempotency logic
```

---

# 9. System Architecture

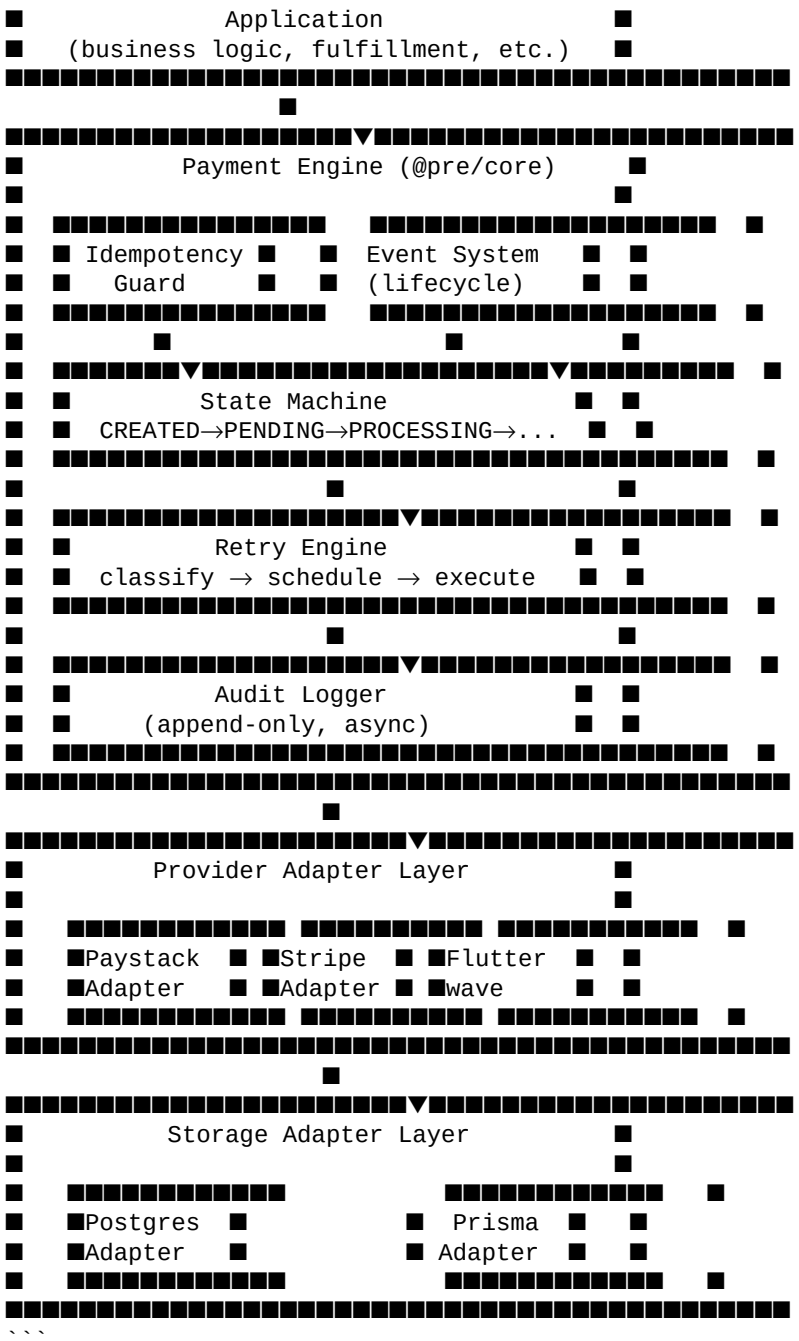
## 9.1 Package Structure

```
@pre/core               → engine, state machine, retry, events, audit
@pre/paystack           → Paystack provider adapter
@pre/stripe             → Stripe provider adapter
@pre/flutterwave        → Flutterwave provider adapter
@pre/postgres           → PostgreSQL storage adapter
@pre/prisma             → Prisma storage adapter
@pre/dashboard          → hosted observability dashboard (SaaS)
```

## 9.2 Layer Architecture

```

```




## 9.3 Webhook Processing Flow

```
Provider sends webhook
        ■
        ▼
Signature verification
  ■■■ FAIL → reject 401, audit log, stop
  ■■■ PASS → continue
        ■
        ▼
Duplicate event check (by event ID)
  ■■■ DUPLICATE → acknowledge 200, audit log, stop
  ■■■ NEW → continue
        ■
        ▼
Find transaction by reference
  ■■■ NOT FOUND → audit log, emit unknown.event, stop
  ■■■ FOUND → continue
        ■
```

```
        ▼
Validate state transition
  ■■■ INVALID → audit log, emit transition.rejected, stop
  ■■■ VALID → continue
        ■
        ▼
Persist new state (atomic)
        ■
        ▼
Write audit entry
        ■
        ▼
Emit lifecycle event to application
        ■
        ▼
Return 200 to provider
```

---

# 10. Public API Reference

## 10.1 Engine Initialization

```ts
import { PaymentEngine } from "@pre/core";
import { PaystackProvider } from "@pre/paystack";
import { PostgresAdapter } from "@pre/postgres";

const engine = new PaymentEngine({
  provider: new PaystackProvider({
    secretKey: process.env.PAYSTACK_SECRET_KEY,
    webhookSecret: process.env.PAYSTACK_WEBHOOK_SECRET,
  }),
  storage: new PostgresAdapter({
    connectionString: process.env.DATABASE_URL,
  }),
  retry: {
    strategy: "exponential",
    maxAttempts: 5,
    initialDelay: 1000,
    maxDelay: 30000,
    jitter: true,
  },
  options: {
    autoReconcile: true,
    reconcileInterval: "0 * * * *", // every hour
    auditAsync: true,
  },
});
```

## 10.2 Initialize Payment

```ts
const result = await engine.initialize({
  amount: 5000,
  currency: "NGN",
  email: "user@example.com",
  idempotencyKey: "order_789_attempt_1",
  metadata: {
    orderId: "order_789",
    userId: "user_123",
    productId: "prod_456",
  },
});
```

## 10.3 Handle Webhook

```ts
app.post("/webhooks/payment", express.raw({ type: "application/json" }), async (req, res) => {
  await engine.handleWebhook({
    payload: req.body,
    signature: req.headers["x-paystack-signature"] as string,
    raw: req.body,
  });
  res.sendStatus(200);
});
```

## 10.4 Verify Payment

```ts
const transaction = await engine.verify(transactionId);
```

## 10.5 Refund Payment

```ts
// full refund
await engine.refund(transactionId);

// partial refund
await engine.refund(transactionId, { amount: 2500 });
```

## 10.6 Reconcile

```ts
// single transaction
await engine.reconcile(transactionId);

// time range
await engine.reconcileRange({
  from: new Date("2026-06-20T00:00:00Z"),
  to: new Date("2026-06-20T23:59:59Z"),
});
```

## 10.7 Event Subscriptions

```ts
engine.on("payment.initialized", (event) => { ... });
engine.on("payment.success", (event) => { ... });
engine.on("payment.failed", (event) => { ... });
engine.on("payment.retrying", (event) => { ... });
engine.on("refund.initiated", (event) => { ... });
engine.on("refund.completed", (event) => { ... });
engine.on("dispute.raised", (event) => { ... });
engine.on("reconciliation.mismatch", (event) => { ... });
engine.on("transition.rejected", (event) => { ... });
```

## 10.8 Audit Trail

```ts
const trail = await engine.getAuditTrail(transactionId);
```

---

# 11. Database Schema

## transactions

```sql
CREATE TABLE pre_transactions (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  reference         VARCHAR(255) UNIQUE NOT NULL,
  idempotency_key   VARCHAR(255) UNIQUE NOT NULL,
  provider          VARCHAR(50) NOT NULL,
  amount            BIGINT NOT NULL,
  currency          CHAR(3) NOT NULL,
  status            VARCHAR(20) NOT NULL DEFAULT 'CREATED',
  email             VARCHAR(255),
  metadata          JSONB DEFAULT '{}',
  provider_data     JSONB DEFAULT '{}',
  created_at        TIMESTAMP DEFAULT NOW(),
  updated_at        TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_pre_transactions_reference ON pre_transactions(reference);
CREATE INDEX idx_pre_transactions_status ON pre_transactions(status);
CREATE INDEX idx_pre_transactions_idempotency ON pre_transactions(idempotency_key);
```

## webhook_events

```sql
CREATE TABLE pre_webhook_events (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id        VARCHAR(255) UNIQUE NOT NULL,
  provider        VARCHAR(50) NOT NULL,
  event_type      VARCHAR(100) NOT NULL,
  transaction_id  UUID REFERENCES pre_transactions(id),
  payload         JSONB NOT NULL,
  processed       BOOLEAN DEFAULT false,
  processed_at    TIMESTAMP,
  created_at      TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_pre_webhook_events_event_id ON pre_webhook_events(event_id);
CREATE INDEX idx_pre_webhook_events_transaction ON pre_webhook_events(transaction_id);
```

## idempotency_keys

```sql
CREATE TABLE pre_idempotency_keys (
  key             VARCHAR(255) PRIMARY KEY,
  transaction_id  UUID NOT NULL REFERENCES pre_transactions(id),
  created_at      TIMESTAMP DEFAULT NOW()
);
```

## audit_logs

```sql
CREATE TABLE pre_audit_logs (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  transaction_id  UUID NOT NULL REFERENCES pre_transactions(id),
  event           VARCHAR(100) NOT NULL,
  previous_state  VARCHAR(20),
  new_state       VARCHAR(20),
  actor           VARCHAR(20) NOT NULL,
  metadata        JSONB DEFAULT '{}',
  created_at      TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_pre_audit_logs_transaction ON pre_audit_logs(transaction_id);
CREATE INDEX idx_pre_audit_logs_created ON pre_audit_logs(created_at);
```

## retries

```sql
CREATE TABLE pre_retries (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  transaction_id   UUID NOT NULL REFERENCES pre_transactions(id),
  attempt_number   INTEGER NOT NULL DEFAULT 1,
  failure_reason   TEXT,
  failure_type     VARCHAR(20) NOT NULL, -- transient | permanent
  next_attempt_at  TIMESTAMP,
  attempted_at     TIMESTAMP,
  succeeded        BOOLEAN,
  created_at       TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_pre_retries_transaction ON pre_retries(transaction_id);
CREATE INDEX idx_pre_retries_next_attempt ON pre_retries(next_attempt_at)
  WHERE succeeded IS NULL;
```

---

# 12. Error Reference

```ts
PreError                      → base error class
  InvalidStateTransitionError → attempted forbidden state transition
  DuplicateIdempotencyError   → idempotency key already used
  WebhookVerificationError    → signature invalid or missing
  ProviderError               → provider returned an error response
  TransactionNotFoundError    → transaction ID does not exist
  RefundError                 → refund failed or invalid
  RetryExhaustedError         → all retry attempts consumed
  StorageError                → persistence operation failed
  ConfigurationError          → engine misconfigured at startup
```

---

# 13. Monetization

## Open Source Core

The `@pre/core` package and all official provider adapters are open source under the MIT licens

## PRE Dashboard (SaaS)

A hosted observability and management platform built on top of the open source engine.

```
Starter    $29/month
  Up to 10,000 transactions/month
  Real-time transaction monitoring
  Failed payment alerts
  Basic reconciliation reports
  7-day audit log retention

Growth     $99/month
  Up to 100,000 transactions/month
  Everything in Starter
  Retry monitoring and control
  Webhook delivery logs
  Provider health status
  30-day audit log retention
  Email support

Scale      $299/month
```

```
  Unlimited transactions
  Everything in Growth
  Custom reconciliation schedules
  Dead-letter queue management
  API access to dashboard data
  90-day audit log retention
  Priority support
  SLA guarantee
```

---

# 14. Roadmap

## Phase 1 — Core (Months 1-2)

```
State machine implementation
Idempotency engine
Paystack provider adapter
Stripe provider adapter
PostgreSQL storage adapter
Webhook processing pipeline
Audit logging
Basic retry engine
npm package publishing
Documentation site
```

## Phase 2 — Ecosystem (Months 3-4)

```
Flutterwave provider adapter
Prisma storage adapter
Reconciliation engine
Partial refund support
Event system
Exponential backoff with jitter
Failure classification
```

## Phase 3 — Dashboard (Months 5-6)

```
PRE Dashboard MVP
Transaction monitoring
Failed payment alerts
Audit log viewer
Reconciliation UI
Webhook delivery logs
```

## Phase 4 — Scale (Months 7-12)

```
Multi-provider failover
Dead-letter queues
Payment analytics
Provider health monitoring
Monnify adapter
LemonSqueezy adapter
Fraud signals
Observability integrations (Datadog, Sentry)
```

---
# 15. Success Metrics

```
Technical
  Duplicate processing rate:    < 0.001%
  Invalid state transitions:    0
  Webhook verification bypass:  0
  Test coverage (core):         > 90%

Adoption
  npm weekly downloads:         > 1,000 within 6 months
  GitHub stars:                 > 500 within 6 months
  Production integrations:      > 50 within 6 months

Business
  Dashboard paying customers:   > 20 within 3 months of launch
  MRR:                          > $1,000 within 3 months of dashboard launch
  Integration time (p50):       < 30 minutes
  Support tickets per user:     < 0.1 per month
```

---

*PRE — Payment Reliability Engine*
*SRD v1.0 — Jason Dagana — June 2026*